

Lab 5 (PCA2-2) Part 1- Index Part 2- PL/SQL OR PROCEDURE

Part 1

index like a **book index** → it helps to find data **fast without scanning full table**.

1. First create table Student (sid, name, dept, age).
2. Now insert at least 10 rows into it.
3. Then retrieve from Students sid and name where dept is 'IT'.
4. Next create an index on Students using the following sql statement
CREATE INDEX idx_dept ON Student(dept);
5. Now execute Qn 3 once again.
6. What is your observation?
With index → index scan
Without → sequential scan
7. Now again repeat actions 3 and 4 by first selecting all names that begin with A
Then create an index on name as in 4. Next again retrieve names that begin with A and note the difference with and without index.
8. CREATE INDEX idx_name ON Student(name);
SELECT * FROM Student WHERE name LIKE 'A%';
9. Note if you do not find any clear observation this is because the table is small.
10. Now you can programmatically with the help of looping insert 100 rows or more into a table.

Part 2 -INTRODUCING PL/SQL or PROCEDURE

PL/SQL is a Procedural Language for SQL

Used in Oracle Database

It is a **full programming language inside the database**

It supports:

- Variables
- Loops
- IF conditions
- Cursors
- Triggers
- Procedures

Example (PL/SQL block)

```
DECLARE
    x INT := 10;
BEGIN
    DBMS_OUTPUT.PUT_LINE(x);
END;
```

What is a Procedure?

A **procedure** is a stored program (function-like block)

It exists in:

- MySQL
- Oracle Database
- PostgreSQL

A typical PL/SQL block

```
DECLARE
  x INT := 10;
BEGIN
  DBMS_OUTPUT.PUT_LINE(x);
END;
```

A procedure in Oracle: which inserts 100 rows in the student table with a for loop

Note Since it is a PL/SQL Procedure it has the **Declare**, **Begin**, and **End** keywords

```
DECLARE
BEGIN
FOR i IN 1..100 LOOP
INSERT INTO Student (sid, name, dept, age)
VALUES (i, 'Student' || i, 'CSE', 18 + MOD(i,5));
END LOOP;
END;
```

The corresponding mysql procedure

```
DELIMITER //

CREATE PROCEDURE insert_students()
BEGIN
DECLARE i INT DEFAULT 1;
WHILE i <= 100 DO
INSERT INTO Student VALUES (i, CONCAT('Student', i), 'CSE', 20);
SET i = i + 1;
END WHILE;
END //

DELIMITER ;

CALL insert_students();
```

More example procedures in both Oracle and Mysql

1. Write a procedure to display all students from the Student table.

Oracle

```
CREATE OR REPLACE PROCEDURE show_students IS
BEGIN
    FOR rec IN (SELECT * FROM Student) LOOP
        DBMS_OUTPUT.PUT_LINE(rec.sid || ' ' || rec.name || ' ' || rec.dept);
    END LOOP;
END;
```

MySQL

```
DELIMITER //

CREATE PROCEDURE show_students()
BEGIN
    SELECT * FROM Student;
END //

DELIMITER ;
```

2. Write a procedure that takes department name as input and displays students of that department.

Oracle

```
CREATE OR REPLACE PROCEDURE get_by_dept(p_dept IN VARCHAR2) IS
BEGIN
    FOR rec IN (SELECT * FROM Student WHERE dept = p_dept) LOOP
        DBMS_OUTPUT.PUT_LINE(rec.name);
    END LOOP;
END;
/
```

MySQL

```
DELIMITER //

CREATE PROCEDURE get_by_dept(IN p_dept VARCHAR(20))
BEGIN
    SELECT name FROM Student WHERE dept = p_dept;
END //

DELIMITER ;
```

3. Write a procedure to count total number of students and display the result.

Oracle

```
CREATE OR REPLACE PROCEDURE count_students IS
    total NUMBER;
BEGIN
    SELECT COUNT(*) INTO total FROM Student;
    DBMS_OUTPUT.PUT_LINE('Total: ' || total);
END;
```

```
END;  
/
```

MySQL

```
DELIMITER //  
  
CREATE PROCEDURE count_students()  
BEGIN  
    SELECT COUNT(*) AS total_students FROM Student;  
END //  
  
DELIMITER ;
```

4. Write a procedure to insert 5 students into the table using a loop.

Oracle

```
CREATE OR REPLACE PROCEDURE insert_students IS  
BEGIN  
    FOR i IN 1..5 LOOP  
        INSERT INTO Student VALUES (i, 'S'||i, 'CSE', 20);  
    END LOOP;  
END;  
/
```

MySQL

```
DELIMITER //  
  
CREATE PROCEDURE insert_students()  
BEGIN  
    DECLARE i INT DEFAULT 1;  
    WHILE i <= 5 DO  
        INSERT INTO Student VALUES (i, CONCAT('S', i), 'CSE', 20);  
        SET i = i + 1;  
    END WHILE;  
END //  
  
DELIMITER ;
```

5. Write a procedure that takes age as input and prints:

- "Adult" if age $\geq$ 18
- "Minor" otherwise

Oracle

```
CREATE OR REPLACE PROCEDURE check_age(p_age IN NUMBER) IS  
BEGIN  
    IF p_age >= 18 THEN  
        DBMS_OUTPUT.PUT_LINE('Adult');  
    ELSE  
        DBMS_OUTPUT.PUT_LINE('Minor');  
    END IF;  
END;  
/
```

MySQL

```
DELIMITER //
```

```
CREATE PROCEDURE check_age(IN p_age INT)
BEGIN
    IF p_age >= 18 THEN
        SELECT 'Adult';
    ELSE
        SELECT 'Minor';
    END IF;
END //
```

```
DELIMITER ;
```

6. Write a procedure that:
 - Takes student id as input
 - Increases age by 1 for that student
7. Write a procedure that finds and displays student(s) having maximum age.
8. Write a procedure that takes sid as input
 - Prints:
 - o "Exists" if student is present
 - o "Not Exists" otherwise
9. Write a procedure that:
 - Inserts N students into the table using a loop.
 - N should be passed as input parameter.
10. Write a procedure that:
 - Displays department-wise student count.